LR-Agent / Agent 后端入门

第04章
Middleware与业务约束

从模型调用前后的一次观察，学会解释阶段门控、执行计划与预算收尾

读者起点与本章边界

前置：已完成第 1、2 章基础实验。只围绕后端输入、决策、执行与验证学习；遇到新名词先看定义和小例子。真实截图、源码阅读图与概念示意分别注明。

学完本章应该能够

- 通过完整实验理解 node hook 与 wrap hook 的顺序
- 沿真实源码解释执行、挂起和抑制三种去向
- 验证未确认不能写、确认后不重复标注的业务规则
- 区分预算、错误反馈和当前未启用的工具重试

源码核对：2026-10-17，工作区基线 d5dd009；本轮包含已验证的日志断管道修复。正式环境 LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与实验准备

4.1 场景：模型提出的调用，都应该执行吗 . . . 3
4.2 两种钩子：站在节点旁，或包住调用 . . . 4
4.3 实验一：观察中间件执行顺序 . . . 5
4.4 顺序会改变行为，不只改变日志 . . . 6
4.5 项目实际装配了哪些策略 . . . 7
4.6 先做计划，再执行：plan_round . . . 8
4.7 阶段来自提案状态的推导 . . . 9
4.8 实验二：等待确认与确认后校验 . . . 10
4.9 允许操作，不等于已经证明成功 . . . 11
4.10 合批与去重：减少请求，保留去向 . . . 12
4.11 实验三：真实回合计划与收尾阈值 . . . 13
4.12 两段式终止：保留本轮合法读取 . . . 14
4.13 预算收尾：限制消耗，也说明进展 . . . 15
4.14 异常反馈与重试的真实边界 . . . 16
4.15 源码实拍：用三处符号串起规则 . . . 17
4.16 练习、项目解释与个人证据 . . . 18
4.17 源码图解：从函数签名读到返回值 . . . 19

每个实验包含全部导入、输入与执行。固定模型和教学数据只验证机制；源码节选只用于阅读。请先预测输出，再运行并改变一个输入。

```
Set-Location D:\Study\LR-Agent
$env:PYTHONPATH = Join-Path (Get-Location) 'LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
$agentPy = 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe'
```

4.1 场景：模型提出的调用，都应该执行吗

第 3 章跑通了工具循环。但用户尚未确认提案时，模型可能又要求标注或生成报告；同一轮还可能重复调用。框架能执行，不代表业务允许执行。业务约束应能根据结构化数据核对。

Middleware（中间件）是在模型或工具调用前后运行的策略代码。它可以观察状态、增加提示、限制调用，或者包装执行来处理结果。它与 HTTP 中间件名字相同，但本章作用位置是 Agent 运行过程。

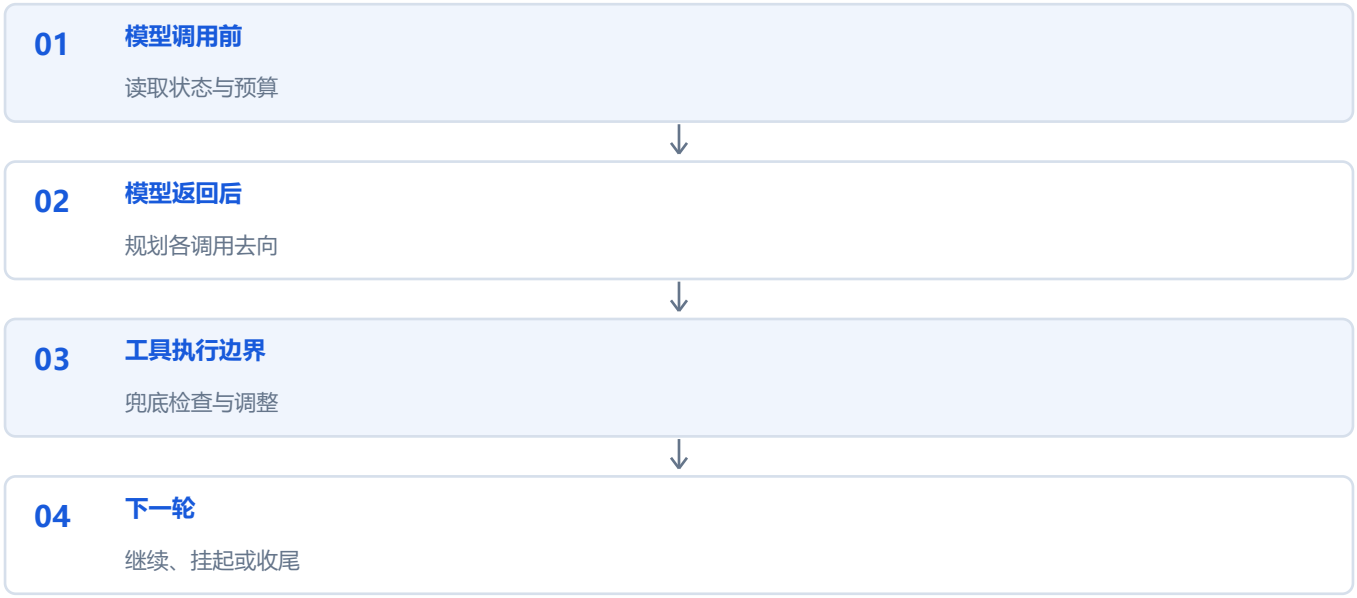


图4-1 概念示意：业务检查进入 Agent 循环

问题	只写提示词的局限	程序检查的事实
未确认就写入	模型可能忽略提醒	提案状态与工具类别
重复标注	模型可能重复规划	路径范围与调用ID
循环不结束	模型未必自行停下	实际轮次与预算

4.2 两种钩子：站在节点旁，或包住调用

Hook（钩子）是框架预留的运行位置。Node hook 在图的某个位置读取 State，可以返回状态更新；wrap hook 接收 request 和 handler，可以在实际调用前后增加逻辑，也可以拒绝继续调用。

钩子	调用位置	常见职责
before_model	进入模型节点前	计数、追加提示
after_model	模型节点完成后	检查调用、更新计划
wrap_model_call	包住模型调用	改请求、处理响应
wrap_tool_call	包住工具调用	门控、异常处理

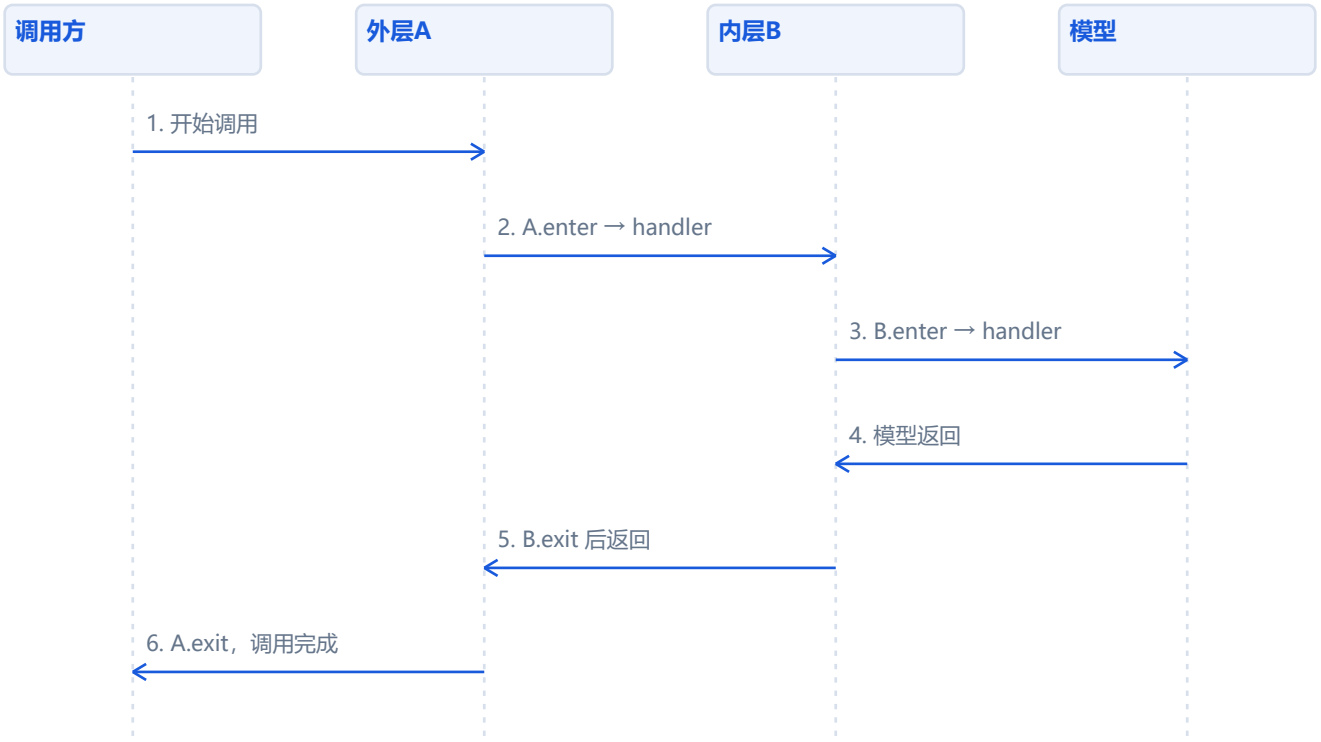


图4-2 概念示意：handler 表示下一层调用

handler 是可调用对象，不是调用结果。handler(request) 才会推进下一层；return response 把结果交回外层。异步版本 awrap_model_call / awrap_tool_call 使用 await handler(request)。

顺序要验证

before 按正序，after 按逆序，wrap 形成嵌套。不要把返回列表从左到右理解成所有钩子的统一执行顺序。

4.3 实验一：观察中间件执行顺序

完整教学实验 ch04_hooks。在 study 自行新建 my_ch04_hooks.py，复制全部代码。

```python
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware
from langchain_core.language_models.fake_chat_models import FakeListChatModel

events = []
class Trace(AgentMiddleware):
    def __init__(self, label):
        self.label = label
    @property
    def name(self):
        return self.label
    def before_model(self, state, runtime):
        events.append(self.label + ".before")
    def after_model(self, state, runtime):
        events.append(self.label + ".after")
    def wrap_model_call(self, request, handler):
        events.append(self.label + ".enter")
        response = handler(request)
        events.append(self.label + ".exit")
        return response

agent = create_agent(FakeListChatModel(responses=["ok"]), [],
                     middleware=[Trace("A"), Trace("B")])
agent.invoke({"messages": [("user", "hello")]})
print(" > ".join(events))
assert events == ["A.before", "B.before", "A.enter", "B.enter",
                  "B.exit", "A.exit", "B.after", "A.after"]
```

运行：& $agentPy ".\study\my_ch04_hooks.py"（工作区根目录）

本轮实际输出：

```
A.before > B.before > A.enter > B.enter > B.exit > A.exit > B.after > A.after
```

Trace 继承 AgentMiddleware；name 属性给两个实例不同名称，避免同名中间件冲突。events 只是实验观察记录，不作为生产并发状态方案。

before 正序 A、B；wrap 进入 A、B，退出 B、A；after 为 B、A。handler 把控制权交给下一层，再沿原路返回。

本例使用框架自带固定模型且没有工具，只验证钩子顺序；不能由此推断项目所有异步业务行为。

4.4 顺序会改变行为，不只改变日志

假设内层重试处理暂时性故障，外层错误件把最终失败变成反馈。若错误件先把异常改成普通结果，重试件便看不到异常。两个 after_model 都返回状态更新时，后执行者也可能影响最终路由。

顺序规则	本实验的证据	项目阅读动作
before 正序	A.before 在 B.before 前	逐个找 before_model
after 逆序	B.after 在 A.after 前	找最后决定路由的件
wrap 首项在外	A.enter 先，A.exit 后	画出 handler 嵌套
异步使用 await	本例仅做同步观察	另对照 awrap 实现

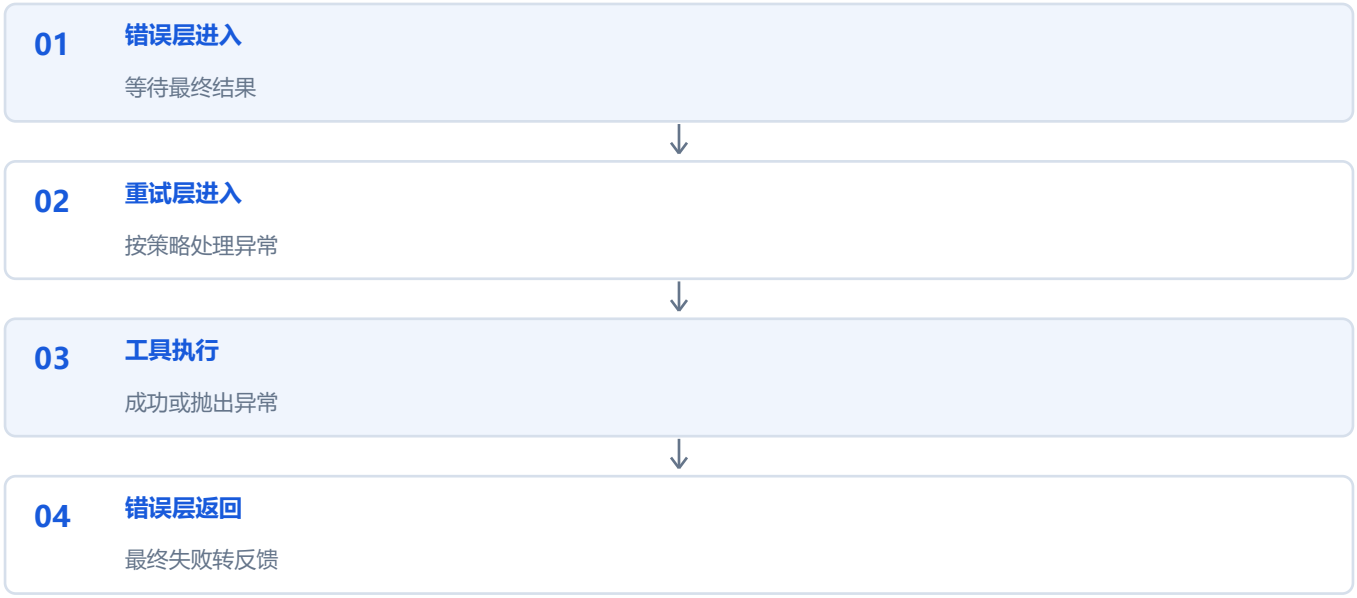


图4-3 概念示意：错误层包住重试层

LR-Agent 把 BusinessRoundMiddleware 放在列表首位，使它的 after_model 在其他相关 after hook 之后落下计划与路由。历史注释存在‘最内层’措辞不一致，教材依据首项为最外层的框架机制与本次实验解释。

不把接入点当启用能力

本页解释相对顺序。当前项目的重试白名单为空，不能因此声称工具会自动重试。

4.5 项目实际装配了哪些策略

先打开 build_business_middleware，按返回列表建立索引。业务回合、模型硬上限、错误、重试、预算收尾之后，才追加 extra。entry.py 当前通过 extra 添加视觉回退与视觉结果处理。

位置	中间件	主要职责
1	BusinessRoundMiddleware	回合计划与执行边界
2	ModelCallLimitMiddleware	模型调用硬上限
3	ToolErrorMiddleware	最终异常转反馈
4	ToolRetryMiddleware	白名单为空，当前不生效
5	BudgetWrapUpMiddleware	无工具模型收尾
extra	两个视觉件	按入口补充视觉能力

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/middleware.py::build_business_middleware

```
ToolRetryMiddleware(
    max_retries=2,
    tools=list(RETRYABLE_TOOL_NAMES),
    retry_on=_retry_on,
    on_failure="error",
    backoff_factor=2.0,
    initial_delay=1.0,
    max_delay=60.0,
    jitter=True,
),
```

max_retries=2 只是配置条件之一，tools 命中才决定具体工具是否适用。RETRYABLE_TOOL_NAMES=() 让任何工具都不进入重试范围。阅读源码时，需要同时看组件配置和启用条件。

4.6 先做计划，再执行：plan_round

plan_round 接收一轮模型消息、阶段上下文和已完成 ID，返回 RoundPlan。它不读写文件，也不执行工具。纯函数便于固定输入检查去向，不必启动完整模型与标注任务。

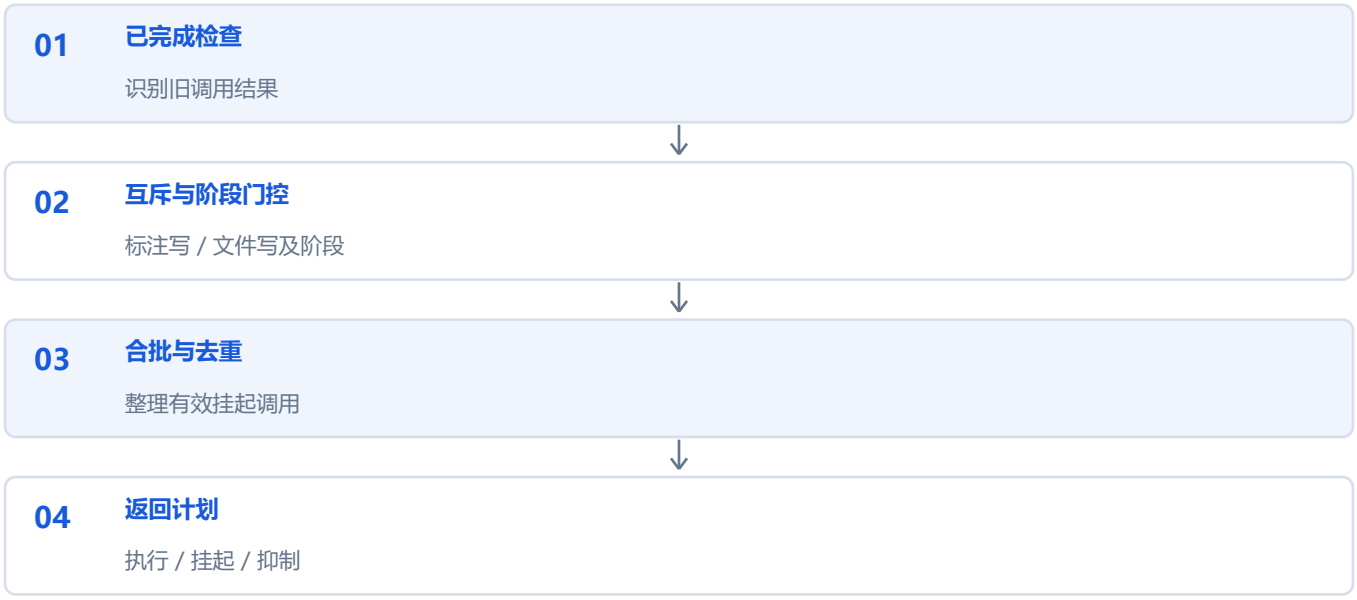


图4-4 项目实际主要决策顺序

RoundPlan 字段	含义	后续消费者
execute_now	本轮 SYNC / PROPOSAL	框架工具节点
deferred	有效外部挂起项	待执行事件适配
suppressed	本轮不该执行的项	合成 ToolMessage
merged_arguments	合并后参数映射	执行包装层改写

合批把多个 auto_annotate 汇成首个调用；去重对相同异步工具与相同路径范围保留首个。两者不同，也不能直接推广为数据库层的 ‘恰好一次’ 保证。

当前短路边界

发现已完成 ID 时，当前函数进入相应短路返回。不要推断 ‘混入旧 ID 仍会执行同轮全部新调用’ ；这种组合应单独核对并测试。

4.7 阶段来自提案状态的推导

用户是否确认本轮提案，会改变允许的下一步动作。derive_task_phase 从 proposal_states 重建 TaskPhaseContext，不依赖后端长期保存阶段变量。pending 表示未确认，applied 表示已确认落盘。

输入状态	推导结果	主要约束
没有提案历史	None	不启用这套阶段门控
有 pending 标注提案	AWAIT_CONFIRM	禁止标注写与文件写
无 pending，有 applied	VERIFY	避免重复整文件标注
只有 dismissed	None	允许重新规划

源码节选：LR-Agent/vendor/local-agent/app/agent/assist/task_phase.py::check_call_allowed

```python
def check_call_allowed(
    name: str,
    arguments: dict[str, Any],
    ctx: TaskPhaseContext | None,
) -> str | None:
```

参数依次是工具名、参数字典和阶段上下文。返回 None 表示本门控允许；返回字符串表示拒绝理由。这不代表路径权限和其他业务检查也全部通过。

不是强制线性图

SCAN、ANNOTATE、REPORT 当前主要作语义标记，硬门控集中在 AWAIT_CONFIRM、VERIFY。源码没有把五个名字编成必须逐站经过的 StateGraph。

4.8 实验二：等待确认与确认后校验

完整教学实验 ch04_phase。在 study 自行新建 my_ch04_phase.py，复制全部代码。

```python
from app.agent.assist.task_phase import derive_task_phase
from app.agent.assist.task_phase import check_call_allowed
from app.schemas.agent import ProposalStateInput

pending = ProposalStateInput(path="./data/a.jpg", status="pending")
context = derive_task_phase([pending])
print("phase:", context.phase.value)
print("read allowed:", check_call_allowed(
    "read_file_annotation", {"paths": ["data/a.jpg"]}, context) is None)
print("write blocked:", check_call_allowed(
    "auto_annotate", {"paths": ["data/a.jpg"]}, context) is not None)
applied = ProposalStateInput(path="data/a.jpg", status="applied")
context = derive_task_phase([applied])
print("next phase:", context.phase.value)
print("repeat blocked:", check_call_allowed(
    "auto_annotate", {"paths": ["data/a.jpg"]}, context) is not None)
print("new path allowed:", check_call_allowed(
    "auto_annotate", {"paths": ["data/b.jpg"]}, context) is None)
assert derive_task_phase([]) is None
```

运行：& $agentPy ".\study\my_ch04_phase.py"（工作区根目录）

本轮实际输出：

```
phase: await_confirm
read allowed: True
write blocked: True
next phase: verify
repeat blocked: True
new path allowed: True
```

ProposalStateInput 是真实请求模型。./data/a.jpg 会归一化；derive_task_phase 依据列表内容推导阶段，不修改业务文件。

等待确认时读取通过、auto_annotate 被拒；改为 applied 后进入 verify，同一路径仍不能整文件重标，明确的新路径通过本层门控。

is None / is not None 用来判断允许与拒绝，而不硬编码提示文字。本例验证规则函数，不证明外部执行方实际落盘。

4.9 允许操作，不等于已经证明成功

实验中 read allowed=True，只表示读取未被阶段规则拦截。等待确认时磁盘通常还是旧状态，因此读取磁盘不能证明待确认提案已经写入。 ‘能不能操作’ 和 ‘结果证明什么’ 必须分别分析。

场景	正确动作	容易犯的错
pending 提案	说明内容，等待确认或关闭	声称已经写盘
applied 后核对	读取真实标注结果	再次要求确认旧提案
已处理路径有错	核对后定向修正	无依据整批重标
verify + 新路径	明确指定目标后再检查	all_files 覆盖全部

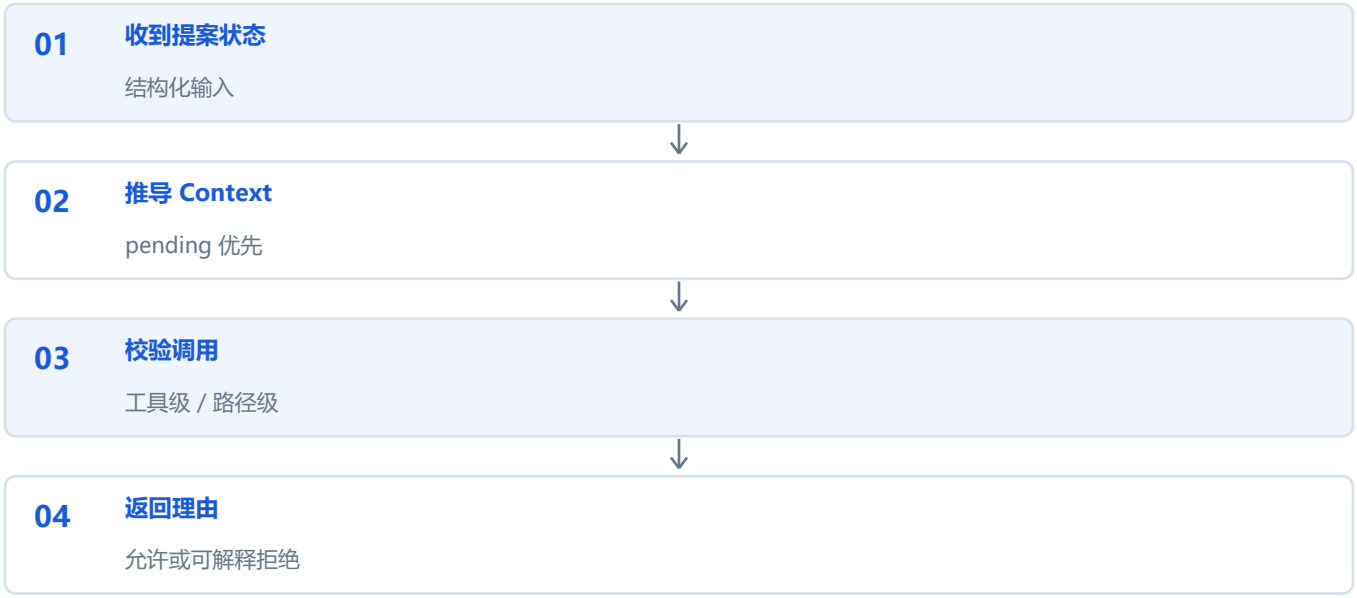


图4-5 阶段约束：确认状态影响下一步

verify 对 auto_annotate 检查范围：all_files=true、未明确路径、与已确认路径重叠，都可能被拒。mutate_annotation 可以带明确路径或标注 ID 定向修正，不能概括成 ‘确认后一切修改禁止’ 。

教学口径

解释可以核对的输入输出，不把阶段名包装成项目并不存在的规划、执行、审查多 Agent 系统。

4.10 合批与去重：减少请求，保留去向

模型同轮返回多个 auto_annotate，项目将路径按首次出现顺序合并到首个调用。其余调用进入 suppressed，返回 coalesced。保留解释比悄悄删除调用更便于追踪和调试。

参数 / 规则	当前实现	注意边界
paths	首次出现顺序的并集	同一路径不重复加入
all_files	任一 True 则 True	仍受授权与门控约束
write_mode	任一 replace_matching 则采用	不是任意参数智能合并
其他字段	沿用首个调用参数	冲突阈值不会自动解决

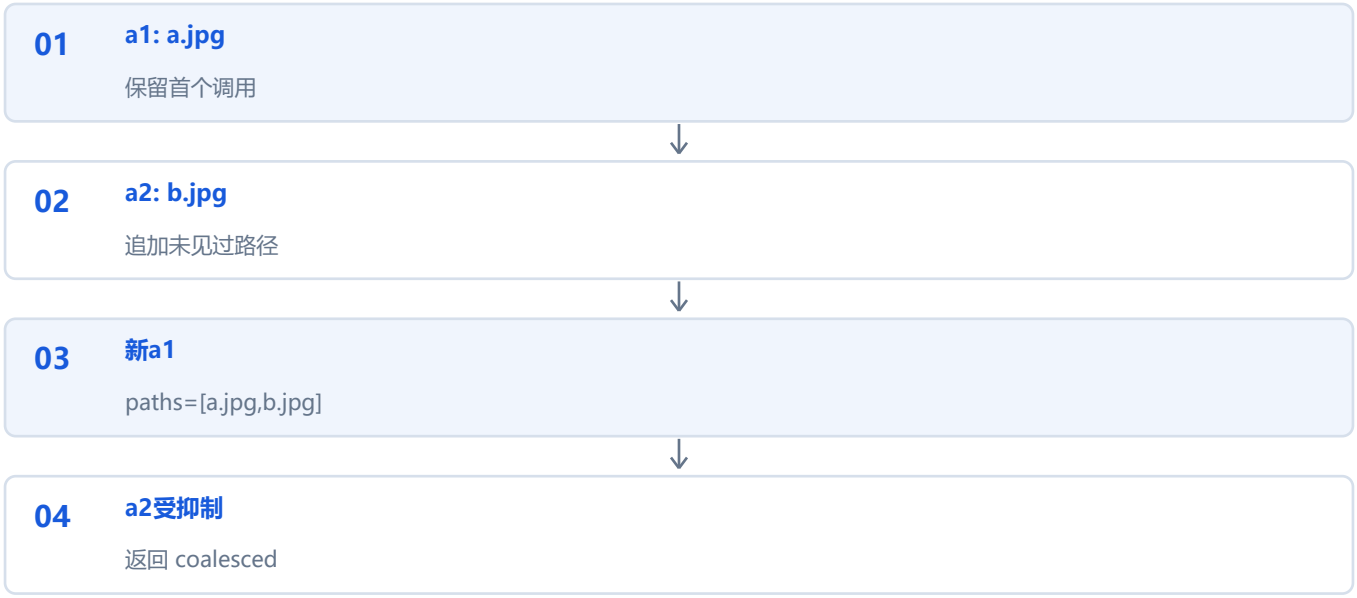


图4-6 概念示意：两个调用形成一个有效挂起项

异步去重用工具名和路径集合构成判定键，不是完整参数比较。因此不能声称所有参数不同的请求都会分别执行。冲突阈值或同路径不同修改细节，都值得单独设例分析。

设计取舍

少调用并非唯一目标。规则还要兼顾副作用、成本与事件重复，并且让每个调用的最终去向可解释。

4.11 实验三：真实回合计划与收尾阈值

完整教学实验 ch04_plan。在 study 自行新建 my_ch04_plan.py，复制全部代码。

```python
from langchain_core.messages import AIMessage
from app.agent.langchain_runtime.round_plan import plan_round
from app.agent.langchain_runtime.middleware import (
    BudgetWrapUpMiddleware, RETRYABLE_TOOL_NAMES,
)

def call(name, call_id, paths):
    return {"name": name, "id": call_id, "args": {"paths": paths}}

message = AIMessage(content="", tool_calls=[
    call("read_file_annotation", "r1", ["data/a.jpg"]),
    call("auto_annotate", "a1", ["data/a.jpg"]),
    call("auto_annotate", "a2", ["data/b.jpg"]),
])
plan = plan_round(message)
print("execute:", [item.tool_call_id for item in plan.execute_now])
print("deferred:", [item.tool_call_id for item in plan.deferred])
print("paths:", plan.deferred[0].arguments["paths"])
print("suppressed:", plan.suppression_reasons)
print("resolved:", plan.all_calls_resolved_here)
assert plan.deferred[0].arguments["paths"] == ["data/a.jpg", "data/b.jpg"]
budget = BudgetWrapUpMiddleware(max_tool_rounds=1)
print("wrap up at:", budget.wrap_up_at)
print("retry names:", RETRYABLE_TOOL_NAMES)
assert budget.wrap_up_at == 3 and RETRYABLE_TOOL_NAMES == ()
```

运行：& $agentPy ".\study\my_ch04_plan.py"（工作区根目录）

本轮实际输出：

```
execute: ['r1']
deferred: ['a1']
paths: ['data/a.jpg', 'data/b.jpg']
suppressed: ('coalesced',)
resolved: True
wrap up at: 3
retry names: ()
```

读取 r1 留在 execute_now；a1 作为有效挂起项携带两个路径；a2 标记 coalesced。这里调用的是项目真实 plan_round。

all_calls_resolved_here=True 表示三个调用都有执行、挂起或抑制归属，不表示工具已经成功执行。计划函数未启动检测。

max_tool_rounds=1 对应收尾阈值 3；重试集合为空。最后断言验证配置事实，未进行完整预算端到端故障注入。

4.12 两段式终止：保留本轮合法读取

一轮中可能同时有合法读取与外部标注调用。如果看到标注就立即结束图，读取也会被跳过。当前策略先规划各调用，让合法 SYNC / PROPOSAL 执行，等回到模型前再结束本轮。

回合情况	after_model	下一步
纯挂起且应结束	计划 + jump_to=end	外层发待执行信息
合法读取 + 挂起	保留读取，标记结束	before_model 再终止
纯阶段拒绝	注入拒绝 ToolMessage	模型理解门控反馈
无工具调用	无需回合计划	按最终回答结束

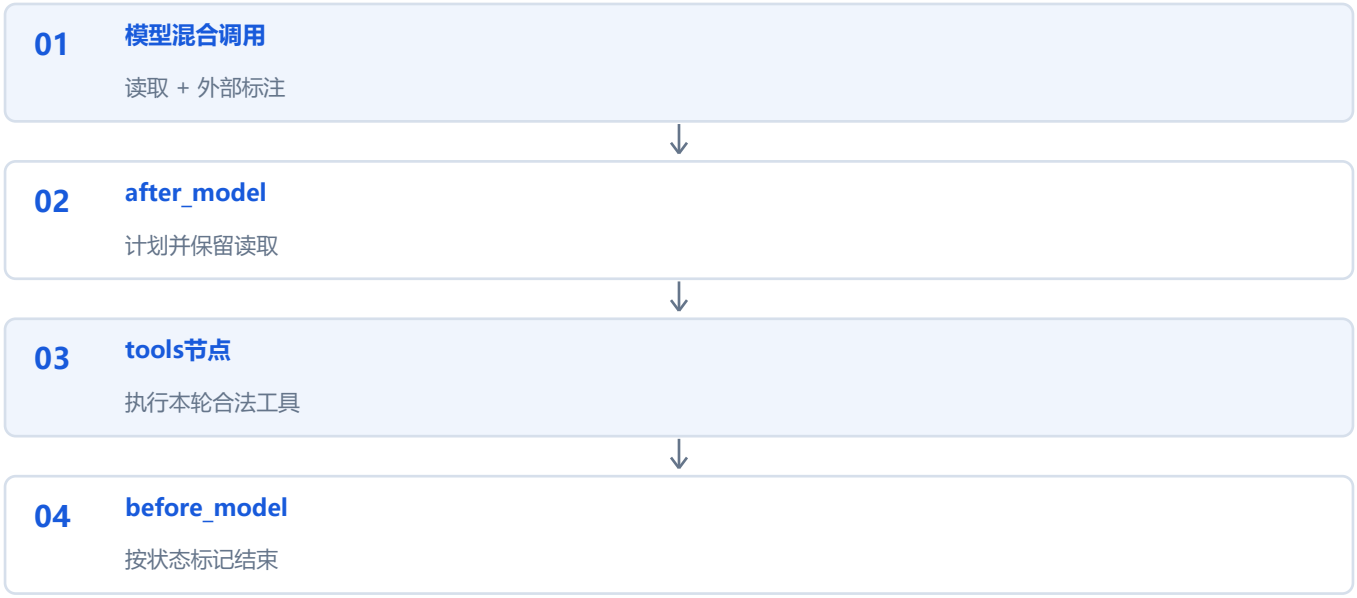


图4-7 项目混合回合：先执行合法项，再结束

受抑制或挂起的调用用相同 tool_call_id 的合成 ToolMessage 表达不进入常规执行。awrap_tool_call 还会无条件拦截 ASYNC，作为执行边界的兜底。计划、路由和门控共同工作。

以当前函数体为准

顶层保留迁移期说明，当前纯挂起分支是 jump_to=end；混合回合需要保留合法执行后的终止步骤。

4.13 预算收尾：限制消耗，也说明进展

预算用来限制循环消耗，不是检测精度阈值。项目同时有模型调用硬上限和业务收尾：前者防止继续调用，后者提供一次无工具总结机会，说明已完成和未完成内容。

max_tool_rounds=1	允许行为	计数解释
模型第1次	正常工具轮	保留旧路径语义
模型第2次	仍是正常工具轮	不是立刻进入收尾
模型第3次	无工具收尾	wrap_up_at=max+2
硬上限	run_limit=3	超出则终止

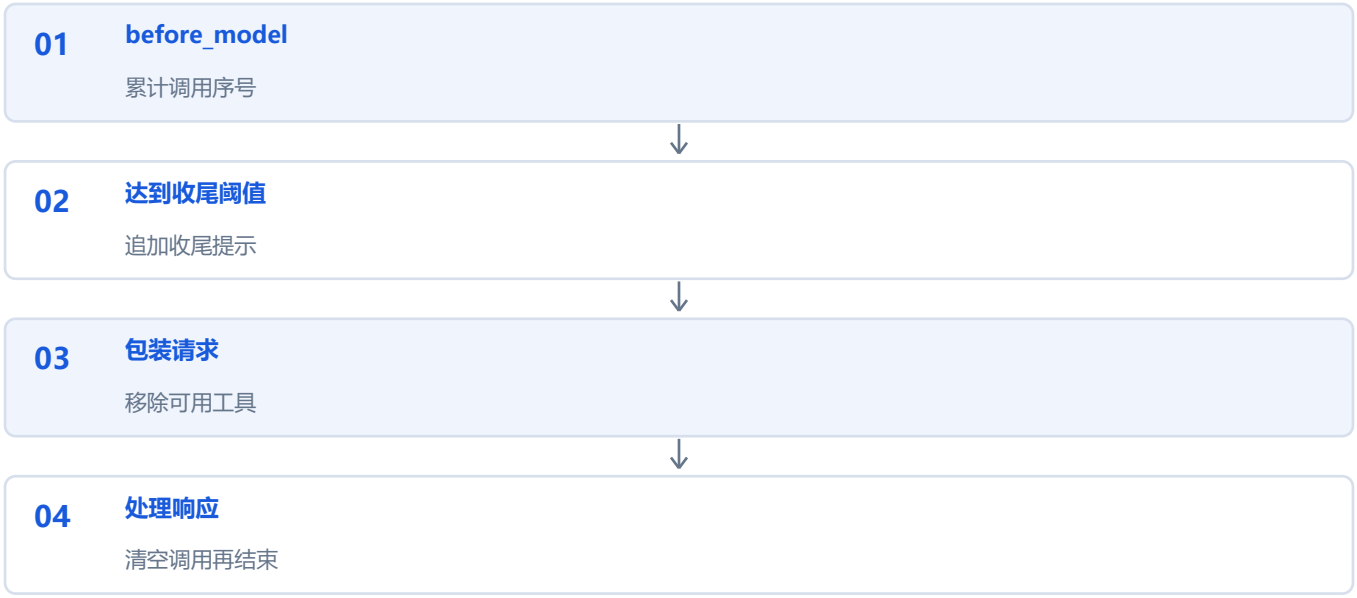


图4-8 项目实际收尾处理

为什么移除工具后还要清空返回调用？模型适配或固定模型仍可能返回工具调用数据。收尾时 request.override(tools=[], tool_choice=None) 移除工具，再由 _blank_tool_calls 同时处理 tool_calls 和 tool_call_chunks。

预算不保证答案正确

收尾只能约束进一步执行。未完成步骤应如实说明，不能预算用尽就把失败写成成功。

4.14 异常反馈与重试的真实边界

Exception（异常）表示调用未正常完成。ToolErrorMiddleware 将适用工具的最终异常转成结构化反馈，供模型和外层协议识别。异常变成消息，并不等于业务已经成功。

机制	当前行为	不能宣称
ToolErrorMiddleware	适用后端工具异常转反馈	所有失败都已修复
ToolRetryMiddleware	工具白名单为空	只读工具全部自动重试
_retry_on	识别连接 / 超时异常	只凭异常类型就重试
外部 ASYNC 工具	后端不执行本体	由本件重试外部副作用

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/middleware.py::RETRYABLE_TOOL_NAMES

```
RETRYABLE_TOOL_NAMES: tuple[str, ...] = ()
```

空白名单是有意取舍。历史故障注入发现重试会放大调用次数，还可能重跑子代理并重复发事件。重新启用前应确认幂等性、次数、耗时和可观察记录；只读不意味着重跑没有代价。

面试中的准确说法

‘保留重试接入点，当前工具白名单为空，我核对了实际启用条件。’ 这比 ‘自动重试保证稳定’ 更经得起源码追问。

4.15 源码实拍：用三处符号串起规则

建议分三次截源码：build_business_middleware 返回列表、after_model 的纯挂起与混合分支、check_call_allowed 的等待确认条件。每次只截相关函数，写下输入、判断和输出。

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/middleware.py::BusinessRoundMiddleware.before_model

```
if state.get("end_after_round") and state.get("round_executed"):
    return {
        "round_executed": False,
        "end_after_round": False,
        "jump_to": "end",
    }
```

定位符号	应解释的输入	应找到的输出
plan_round	AIMessage 与阶段	RoundPlan
after_model	消息与 State	计划、消息与路由
awrap_tool_call	具体调用 request	拦截或 handler 结果
before_model	回合完成标记	结束或继续

@hook_config 声明允许的跳转目标，返回 jump_to 才能按条件边路由。只往字典放一个字段，不一定改变执行流程，需要核对框架配置与实际图行为。

证据分层

钩子与规则实验是本轮执行；生产装配来自源码核对；真实标注闭环属于已有验收。本章未重跑完整真实模型任务。

4.16 练习、项目解释与个人证据

练习一：阶段实验加入 status='dismissed' 的提案，先预测再运行。参考思路：只有关闭的标注提案时返回 None，只代表不启用这套门控，不代表获得任意文件权限。

练习二：将第二个 auto_annotate 也改成 data/a.jpg。参考思路：先进行合批，路径并集只有一个文件；输入重复不代表状态一定是 duplicate_call，要按真实决策顺序分析。

面试任务	回答主线	个人证据
30秒解释Middleware	模型 / 工具前后策略位置	钩子顺序输出
2分钟讲业务约束	提案→计划→执行兜底	阶段与合批实验
为什么不全自动重试	副作用、放大、重复事件	空白名单源码
怎样定位失败	阶段、ID、计划和预算	输入与失败输出

2 分钟项目说明：未确认不能写 → 结构化状态推导阶段 → 纯函数生成回合计划 → middleware 路由和兜底 → 实验验证边界。先说业务问题，再介绍类名。

不虚构个人贡献

完成本章可记录‘复现门控与合批，解释中间件顺序及重试边界’。若写个人改造成果，还需具体缺陷、本人提交、前后行为和回归证据；已有功能与测试数不是本人新增成果。

4.17 源码图解：从函数签名读到返回值

先看图中的真实路径与原始行号，再打开源文件跟读。签名说明输入条件，函数体说明处理，return说明交给下一层的数据。图为源码阅读辅助，不替代完整上下文。

真实源码阅读图 / 完整中间件返回列表

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/middleware.py

```
596 def build_business_middleware(
597     *,
598     max_tool_rounds: int,
599     phase_ctx: TaskPhaseContext | None = None,
600     completed_tool_ids: frozenset[str] = frozenset(),
601     phase_ctx_getter: Callable[[Any], TaskPhaseContext | None] | None = None,
602     end_turn_on_deferred: bool = True,
603     extra: Sequence[AgentMiddleware] = (),
604 ) -> list[AgentMiddleware]:
... # 文档说明省略，以下保留原始行号
640     return [
641         BusinessRoundMiddleware(
642             phase_ctx=phase_ctx,
643             completed_tool_ids=completed_tool_ids,
644             phase_ctx_getter=phase_ctx_getter,
645             end_turn_on_deferred=end_turn_on_deferred,
646         ),
647         ModelCallLimitMiddleware(run_limit=max_tool_rounds + 2, exit_behavior="end"),
648         ToolErrorMiddleware(
649             on_error=_old_error_text,
650             aon_error=_old_error_text_async,
651             tools=list(SYNC_TOOL_NAMES),
652         ),
653         ToolRetryMiddleware(
654             max_retries=2,
655             tools=list(RETRYABLE_TOOL_NAMES),
656             retry_on=_retry_on,
657             on_failure="error",
658             backoff_factor=2.0,
659             initial_delay=1.0,
660             max_delay=60.0,
661             jitter=True,
662         ),
663         BudgetWrapUpMiddleware(max_tool_rounds=max_tool_rounds),
664         *extra,
665     ]
```

真实源码阅读图：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/middleware.py::build_business_middleware。按当前文件原始行号截取并排版生成。

先看什么	怎样读	自己记录什么
输入	区分实参、配置与上下文	哪些字段可缺失
处理	找条件、调用和错误分支	哪一步改变消息或执行
输出	确认返回值与接收者	怎样用实验检查

源码与证据索引

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/middleware.py::build_business_middleware / BusinessRoundMiddleware / BudgetWrapUpMiddleware / RETRYABLE_TOOL_NAMES

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/round_plan.py::RoundPlan / plan_round / _coalesce_auto_annotate

LR-Agent/vendor/local-agent/app/agent/assist/task_phase.py::derive_task_phase / check_call_allowed

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::build_engine_agent

LR-Agent/vendor/local-agent/app/schemas/agent.py::ProposalStateInput

阅读时先定位符号，再看完整输入、处理与返回。源码存在、离线实验通过和已有真实验收属于不同证据；未验证的模型质量、效率与并发结论仍需单独实验。